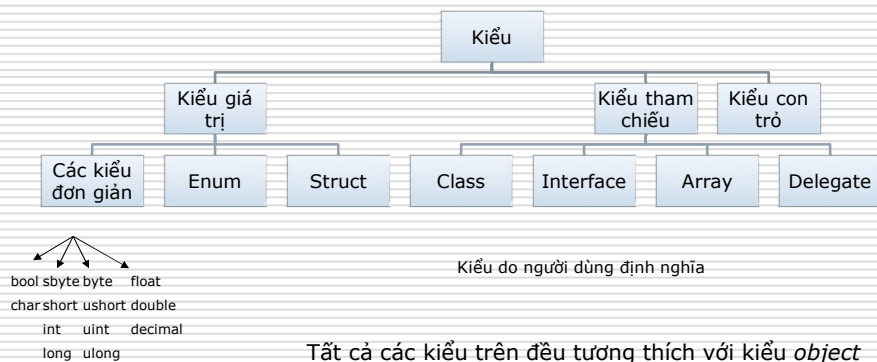


KIỂU DỮ LIỆU & TOÁN TỬ

1

1

PHÂN LOẠI KIỂU



Tất cả các kiểu trên đều tương thích với kiểu *object*

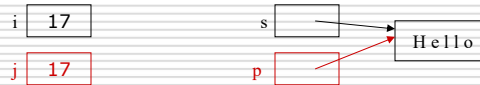
- Có thể được gán với các biến có kiểu *object*
- Các toán tử của kiểu *object* có thể được áp dụng trên chúng

2

2

KIỂU GIÁ TRỊ & KIỂU THAM CHIỂU

	Kiểu giá trị	Kiểu tham chiếu
Biến chứa	Giá trị	Địa chỉ tham chiếu
Nơi lưu trữ	Stack	Heap
Giá trị khởi tạo	0, false, '\0'	null
Lệnh gán	Sao chép giá trị	Sao chép tham chiếu
Ví dụ	int i = 17; int j = i;	string s = "Hello" string p = s;



3

3

CÁC KIỂU DỰNG SẴN

Kiểu	Kế thừa từ	Kích thước	Mô tả - giá trị
byte	System.Byte	1	Không dấu (0..255)
char	System.Char	2	Mã ký tự Unicode
bool	System.Boolean	1	true hay false
sbyte	System.SByte	1	Có dấu (-128 .. 127)
short	System.Int16	2	Có dấu (-32768 .. 32767)
ushort	System.UInt16	2	Không dấu (0 .. 65535)
int	System.Int32	4	Có dấu (-2147483647 .. 2147483647)
uint	System.UInt32	4	Không dấu (0 .. 4294967295)
float	System.Single	4	Số thực ($\approx \pm 1.5 \cdot 10^{-45} \dots \approx \pm 3.4 \cdot 10^{38}$)
double	System.Double	8	Số thực ($\approx \pm 5.0 \cdot 10^{-324} \dots \approx \pm 1.7 \cdot 10^{308}$)
decimal	System.Decimal	8	số có dấu chấm thập phân với 28 ký số và dấu chấm
long	System.Int64	8	Số nguyên có dấu ($-2^{63} \dots 2^{63}-1$)
ulong	System.UInt64	8	Số nguyên không dấu (0 .. $2^{64}-1$)

4

4

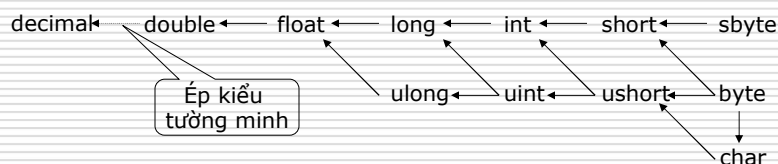
Các ký tự đặc biệt thông dụng

Ký tự	Nghĩa
\'	dấu nháy đơn
\"	dấu nháy đôi
\\	dấu chéo ngược "\"
\0	Null
\a	Alert
\b	lùi về sau
\f	Form feed
\n	xuống dòng
\r	về đầu dòng
\t	Tab ngang
\v	Tab dọc

5

5

CHUYỂN ĐỔI KIỂU



□ Một đối tượng có thể chuyển đổi từ kiểu này sang kiểu khác theo hai cách:

- Chuyển đổi ngầm định: được thực hiện tự động bởi trình biên dịch.
- Chuyển đổi tường minh: có sự can thiệp của người lập trình.

```
//Ép kiểu ngầm định
short x = 10;
int y = x;
```

```
//Lỗi ép kiểu
int y = 10;
short x = y;
```

```
//Ép kiểu tường minh
int y = 10;
short x = (short) y;
```

6

6

KIỂU LIỆT KÊ : enum

- Được sử dụng để liệt kê các hằng

Khai báo

```
enum Color {Red, Blue, Green} // giá trị mặc định: 0, 1, 2
enum Ngay {Hai=2, Ba, Tu, Nam, Sau, Bay, Chunhat=10}
enum Thang:byte {Mot, Hai=2, Ba, Tu, Nam, Sau, Bay=10}
```

Sử dụng

```
Color c = Color.Blue;
int x = (int) Ngay.Hai;
int y = (int) Ngay.Sau
byte z = (byte) Thang.Ba;
```

7

7

TOÁN TỬ KIỂU ENUM

Toán tử	Ví dụ
So sánh	if (c == Color.Red) ... if (c > Color.Red && c <= Color.Green) ...
+, -	c = c + 2;
++, --	c++; c--
&	if ((c & Color.Red) == 0) ...
	c = c Color.Blue;
~	c = ~ Color.Red;

- Chương trình không kiểm tra kết quả có phải là một giá trị hợp lệ của kiểu enum không
- Chú ý:
 - enum không được gán cho kiểu int, trừ khi phải ép kiểu tường minh
 - enum kế thừa kiểu object (Equals, ToString, ...)
 - Lớp System.enum cung cấp các phương thức : GetName, Format, GetValues,...

8

8

MỘT SỐ KIỂU MỞ RỘNG

- Kiểu xâu (string) : được sử dụng như một kiểu chuẩn string.
 - Khai báo : `string s = "Hello";`
 - Kiểu string không thể chỉnh sửa được (Nếu muốn chỉnh sửa dùng `StringBuilder`)
 - Toán tử nối chuỗi là phép cộng `"+"`
 - string là kiểu tham chiếu, nhưng có thể thực hiện phép so sánh
 - Lớp string có nhiều phương thức hữu dụng: `Length`, `CompareTo`, `IndexOf`, `StartsWith`, `Substring`...

9

9

MỘT SỐ KIỂU MỞ RỘNG

- Mảng : tập hợp các phần tử có cùng kiểu
 - Khai báo : `int [] a; int [][] b;`
 - Mỗi thành phần được xác định thông qua chỉ số
 - Có nhiều kiểu mảng : mảng một chiều, mảng nhiều chiều, mảng ziczắc, ...
- Danh sách : mảng có kích thước thay đổi.
 - Khai báo : `ArrayList a;`
- Cấu trúc (struct)
- Lớp (class)

10

10

BIẾN VÀ HẲNG

- Biến là một vùng nhớ dành riêng cho một kiểu dữ liệu nào đó.
 - Biến có thể được gán giá trị và có thể thay đổi giá trị trong quá trình thực hiện chương trình.
 - Biến phải luôn được gán giá trị trước khi được sử dụng.
 - Cú pháp khai báo biến
 - <tên kiểu> <tên biến>
 - Ví dụ : `int x,y;`
- Hằng là một biến nhưng giá trị của hằng không thể thay đổi trong quá trình thực hiện chương trình.
 - Cú pháp khai báo hằng
 - <const> <tên kiểu> <tên hằng> = <giá trị>
 - Ví dụ : `const float so_pi = 3.1415`

11

11

TOÁN TỬ

- Toán tử được ký hiệu bằng một biểu tượng, dùng để thực hiện một hành động
- Toán tử gán: được ký hiệu là dấu "=", là toán tử hai ngôi làm toán hạng bên trái có giá trị bằng toán hạng bên phải.
 - Ví dụ : `x = y;`
- Toán tử toán học : Là các toán tử thao tác số học.
 - Có 5 toán tử toán học : +, -, *, / và %
- Toán tử tăng, giảm: Thực hiện các thao tác số học và gán lại giá trị cho chính biến được tính toán
 - Có 5 toán tử tăng, giảm tương ứng với 5 toán tử số học: +=, -=, *=, /=, %=
 - Ví dụ : `x = x + 3` tương đương với `x += 3;`

12

12

TOÁN TỬ

- Toán tử tăng, giảm 1: ++, --
 - Ví dụ: $x = x + 1$ tương đương với $x+=1$ hoặc $x++$
 - Có hai loại tăng, giảm 1 :
 - Tăng, giảm tiền tố : $++x$ hoặc $--x$
 - Tăng, giảm hậu tố : $x++$ hoặc $x--$
- Toán tử quan hệ : Dùng để so sánh giữa hai giá trị và trả về một giá trị logic kiểu bool (true hoặc false).
 - $==$: so sánh bằng
 - $!=$: so sánh khác
 - $>$: so sánh lớn hơn
 - $>=$: so sánh lớn hơn hoặc bằng
 - $<$: so sánh nhỏ hơn
 - $<=$: so sánh nhỏ hơn hoặc bằng

13

13

TOÁN TỬ

- Toán tử logic : Dùng để kết hợp hai hoặc nhiều biểu thức logic
 - $\&\&$: kết hợp theo phép and
 - $||$: kết hợp theo phép or
 - $!$: kết hợp theo phép not.
- Toán tử thao tác bit : Dùng để thực hiện các phép toán nhị phân
 - $\&$: phép and bit
 - $|$: phép or bit
 - $\sim$: phép not bit
 - $\wedge$: phép xor bit
- Toán tử ba ngôi :
 - Cú pháp : $<\text{biểu thức điều kiện}> ? <\text{biểu thức 1}> : <\text{biểu thức 2}>$
 - Nếu biểu thức điều kiện đúng sẽ trả về giá trị của biểu thức 1, ngược lại trả về giá trị của biểu thức 2.

14

14

ĐỘ ƯU TIÊN TOÁN TỬ

STT	Loại toán tử	Toán tử	Thứ tự
1	Phép toán cơ bản	(x) x.y f(x) a[x] x++ x--	Trái sang phải
2	Toán tử một ngôi	+ - ! ~ ++x --x (T)x	Trái sang phải
3	Phép nhân	* / %	Trái sang phải
4	Phép cộng	+ -	Trái sang phải
5	Dịch bit	>> <<	Trái sang phải
6	Quan hệ	> < >= <= is	Trái sang phải
7	So sánh bằng	= = !=	Phải sang trái
8	Phép toán AND bit	&	Trái sang phải
9	Phép toán XOR bit	^	Trái sang phải
10	Phép toán OR bit		Trái sang phải
11	Phép toán logic AND	&&	Trái sang phải
12	Phép toán logic OR		Trái sang phải
13	Điều kiện	? :	Phải sang trái
14	Phép gán	= += -= *= /= <<= >>= &= ^= =	Phải sang trái